

Virtual Karaoke Implementation Final Summary

Group 26: Vinny Manara, Jinghui Pang, Svastik Patel

The Virtual Karaoke project implements a real-time, browser-based karaoke platform that allows users to join shared rooms, stream audio and video through WebRTC, chat with other participants, and perform synchronized karaoke tracks. The system recreates the social experience of in-person karaoke by supporting a shared queue, synchronized playback, and multi-user video grids. The project domain centers on remote entertainment technologies, addressing the limitations of general video-calling tools that lack coordinated playback and performance-oriented interaction. The implementation is grounded in the design team's earlier documents, including the Virtual Karaoke Final Report and Scenario 1 and Scenario 2 specifications, which defined the functional scope for room creation, device testing, song search, queue management, and synchronized playback. Naming conventions, UML notation, and a structured data dictionary ensured consistency across the system's components.

The project deliverables were completed in two incremental releases. The first release implemented Scenario 1, providing room creation, room entry, chat functionality, and camera/microphone verification using browser media APIs. This release also established the initial Socket.io signaling structure and room state management. The second release implemented Scenario 2, adding the searchable song library, shared queue, synchronized playback, multi-user video streaming, and upvoting features. A comparison with the original design document highlights several deviations, such as using SQLite instead of MySQL, React and Python instead of C/C++, and yt-dlp extraction instead of direct integration with commercial music platforms. These changes were made to simplify development while still meeting core functional requirements.

Testing focused on the three major components of the system: the Room Component, the Node.js signaling server, and the FastAPI song service. The Room Component required extensive testing due to its complexity, including WebRTC peer connections, playback synchronization, and queue updates. The signaling server was tested for event handling, message validation, and consistent state propagation. The song service was tested for correct search results, SQL behavior, and media extraction reliability. Integrated system behavior, such as multi-user video grids, synchronized playback, and chat messaging, was also tested to ensure that all components worked together as intended.

Test specifications were defined for six core tests: room joining and WebRTC initialization, synchronized playback, queue update propagation, song search, YouTube stream extraction, and mic/camera toggle behavior. These tests validated the system's ability to establish peer connections, synchronize playback actions, maintain consistent queue state, return accurate API results, extract playable media streams, and propagate track-state changes across clients. Test results showed that most components behaved as expected, though playback

synchronization and mic/camera toggling exhibited inconsistencies under certain conditions, particularly in larger rooms or during seek operations. These issues were documented and addressed where possible.

Regression testing was performed on components that initially failed or exhibited inconsistent behavior. Playback synchronization was re-tested after timing adjustments, showing improvement but still minor drift under network latency. Queue update consistency was fully resolved after eliminating redundant server emissions. Media extraction reliability improved after correcting format selection in yt-dlp. These regression tests confirmed that fixes did not introduce new issues and helped identify areas requiring further refinement. The inspection phase further validated the system through structured code reviews, covering the Room Component, signaling server, and song service. Inspection meetings conducted via Discord allowed the team to walk through code collaboratively, identify race conditions, correct SQL errors, and resolve state-management issues.

The final sections of the report document recommendations, project issues, and future enhancements. Open issues include video resetting when new users join, inconsistent mic/camera toggling in larger rooms, and desynchronization during seek operations. The waiting room records deferred features such as a lobby system, private room codes, song preview, user statistics, and a “mute all” mode. Ideas for solutions include adopting more robust synchronization frameworks, integrating TURN servers, using TypeScript for stronger type safety, and containerizing the system with Docker. The project retrospective reflects on challenges such as last-minute coding and early communication gaps, which improved through shared calendars and collaborative debugging sessions. Overall, the Virtual Karaoke project delivers a functional multi-user karaoke platform and establishes a strong foundation for future development.